

Struct

结构体

结构体 (struct) , 可以看做是一系列称为成员元素的组合体。

可以看做是自定义的数据类型。

▼ Note

本页描述的 struct 不同于 C 中 struct, 在 C++ 中 struct 被扩展为类似 [class](#) 的类说明符。

定义结构体

```
1 struct Object {
2     int weight;
3     int value;
4 } e[array_length];
5
6 const Object a;
7 Object b, B[array_length], tmp;
8 Object *c;
```

上例中定义了一个名为 Object 的结构体, 两个成员元素 value, weight, 类型都为 int。

在 } 后, 定义了数据类型为 Object 的常量 a, 变量 b, 变量 tmp, 数组 B, 指针 c。对于某种已经存在的类型, 都可以使用这里的方法进行定义常量、变量、指针、数组等。

关于指针: 不必强求掌握。

定义指针

如果是定义内置类型的指针, 则与平常定义指针一样。

如果是定义结构体指针, 在定义中使用 StructName* 进行定义。

```
1 struct Edge {
2     /*
3     ...
4     */
5     Edge* nxt;
6 };
```

上例仅作参考, 不必纠结实际意义。

访问/修改成员元素

可以使用 变量名.成员元素名 进行访问。例如可以使用 cout << var.v 来输出 var 的 v 成员。

也可以使用 指针名->成员元素名 或者 使用 (*指针名).成员元素名 进行访问。例如使用 (*ptr).v = tmp 或者 ptr->v = tmp 可以将结构体指针 ptr 指向的结构体的成员元素 v 赋值为 tmp: 。

为什么需要结构体?

首先，条条大路通罗马，可以不使用结构体达到相同的效果。但是结构体能够显式地将成员元素（在算法竞赛中通常是变量）捆绑在一起，如本例中的 `Object` 结构体，便将 `value, weight` 放在了一起（定义这个结构体的实际意义是表示一件物品的重量与价值）。这样的好处边是限制了成员元素的使用。

想象一下，如果不使用结构体而且有两个数组 `value[], Value[]`，很容易写混淆。但如果使用结构体，能够减轻出现使用变量错误的几率。

并且不同的结构体（结构体类型，如 `Object` 这个结构体）或者不同的结构体变量（结构体的实例，如上方的 `e` 数组）可以拥有相同名字的成员元素（如 `tmp.value, b.value`），同名的成员元素相互独立（拥有各自的内存，比如说修改 `tmp.value` 不会影响 `b.value` 的值）。

这样的好处是可以使用尽可能相同或者相近的变量去描述一个物品。比如说 `Object` 里有 `value` 这个成员变量；我们还可以定义一个 `Car` 结构体，同时也拥有 `value` 这个成员；如果不使用结构体，或许我们就需要定义 `valueOfObject[], valueOfCar[]` 等不同名称的数组来区分。

如果想要更详细的描述一种事物，还可以定义成员函数。请参考 [类](#) 获取详细内容。

更多的操作？

详见 [类](#)。

注意事项

为了访问内存的效率更高，编译器在处理结构中成员的实际存储情况时，可能会将成员对齐在一定的字节位置，也就意味着结构中有空余的地方。因此，该结构所占用的空间可能大于其中所有成员所占空间的总和。

参考资料

1. [Class - zh.cppreference.com](#)
2. [Data structures - cplusplus.com](#)
3. [对齐方式 - Microsoft Docs](#)

本页面最近更新：2023/12/19 13:50:33, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [lr1d](#), [Lans1ot](#), [cjsoft](#), [Enter-tainer](#), [kLiHz](#), [ksyx](#), [Molmin](#), [Tiphereth-A](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用